

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings

warnings.filterwarnings('ignore')
```

Metodología y datos

Fuente y diseño de la base analítica

```
In [2]: datos_hogar = pd.read_csv('../datos_originales/EPFhogar_2023.tab', sep='\t',
                                index_col=0)
datos_gastos = pd.read_csv('../datos_originales/EPFgastos_2023.tab', sep='\t',
                             index_col=0)

rename_map = {
    'NUMERO': 'id',
    'FACTOR': 'factor',
    'CCAA': 'c_autonoma',
    'TAMAMU': 'tam_municipio',
    'UC1': 'tamano_ocde',
    'NMIEM5': 'n_menores',
    'NMIEM6': 'n_mayores',
    'NUMOCU': 'n_ocupados',
    'SEXOSP': 'hombre',
    'EDADSP': 'edad',
    'OCUSP': 'ocupado',
    'ESTUDIOSSP': 'estudios',
    'SECTOR': 'sector_publico',
    'IMPEXAC': 'ingresos_hogar',
    'REGTEN': 'alquiler',
    'NHABIT': 'n_habitaciones',
    'TIPHOGAR7': 'tipo_hogar'
}

cols_keep = [
    'id', 'factor', 'c_autonoma', 'tam_municipio',
    'tamano_ocde', 'n_menores', 'n_mayores', 'n_ocupados',
    'hombre', 'edad', 'ocupado', 'estudios', 'sector_publico',
    'ingresos_hogar', 'alquiler', 'n_habitaciones',
    'tipo_hogar'
]

datos_hogar = datos_hogar.rename(columns=rename_map)[cols_keep]
```

```
In [3]: print(datos_hogar.dtypes)
```

```

id                int64
factor            float64
c_autonoma        int64
tam_municipio     int64
tamano_ocde       float64
n_menores         int64
n_mayores         int64
n_ocupados        int64
hombre            int64
edad              int64
ocupado           int64
estudios          int64
sector_publico    float64
ingresos_hogar    int64
alquiler          int64
n_habitaciones    int64
tipo_hogar        int64
dtype: object

```

factor

No tiene -9 y no hay que recodificarla.

```
In [4]: print(datos_hogar["factor"].isin([-9, np.nan]).any())
```

False

c_autonoma

No tiene -9, la pasamos a categórica con sus respectivas etiquetas.

```
In [5]: print(datos_hogar["c_autonoma"].isin([-9, np.nan]).any())
```

False

```
In [6]: labels_c_autonoma = {1: 'Andalucía', 2: 'Aragón', 3: 'Asturias', 4: 'Islas Balea
6: 'Cantabria', 7: 'Castilla y León', 8: 'Castilla-La Manch
10: 'Comunidad Valenciana', 11: 'Extremadura', 12: 'Galicia
14: 'Murcia', 15: 'Navarra', 16: 'País Vasco', 17: 'La Rioja
18: 'Ceuta', 19: 'Melilla'}
datos_hogar['c_autonoma'] = datos_hogar['c_autonoma'].map(labels_c_autonoma).ast

```

tam_municipio

No tiene -9, y la pasamos a categórica

```
In [7]: print(datos_hogar["tam_municipio"].isin([-9, np.nan]).any())
```

False

```
In [8]: # Municipios pequeños: Menosde 20,000 habitantes
# Municipios medianos: Entre 20,000 y 49,999 habitantes
# Municipios grandes: Entre 50,000 y 99,999 habitantes
# Municipios muy grandes: 100,000 o más habitantes

labels_tam_municipio = {1: 'Muy Grande', 2: 'Grande', 3: 'Mediano',
4: 'Pequeño', 5: 'Pequeño'}
datos_hogar['tam_municipio'] = datos_hogar['tam_municipio'].map(labels_tam_munic

```

tamano_ocde

No tiene -9 y no necesita ser recodificada

```
In [9]: print(datos_hogar["tamano_ocde"].isin([-9, np.nan]).any())
```

False

n_menores

No tiene -9 y no necesita ser recodificada

```
In [10]: print(datos_hogar["n_menores"].isin([-9, np.nan]).any())
```

False

n_mayores

No tiene -9 y no necesita ser recodificada

```
In [11]: print(datos_hogar["n_mayores"].isin([-9, np.nan]).any())
```

False

n_ocupados

No tiene -9 y no necesita ser recodificada

```
In [12]: print(datos_hogar["n_ocupados"].isin([-9, np.nan]).any())
```

False

hombre

No tiene -9 y establecemos:

- **1** si el sustentador de la vivienda es un hombre, 0 en otro caso.
- **0** en otro caso.

```
In [13]: print(datos_hogar["hombre"].isin([-9, np.nan]).any())
```

False

```
In [14]: datos_hogar['hombre'] = np.where(datos_hogar['hombre'] == 1, 1, 0)
```

edad

No tiene -9 y pero necesita ser recodificada porque el 85 significa 85 o más

```
In [15]: print(datos_hogar["edad"].isin([-9, np.nan]).any())
```

False

```
In [16]: bins = [0, 30, 45, 65, 120]
labels = ['18-30', '31-45', '46-65', 'Más de 65']

datos_hogar['edad'] = pd.cut(datos_hogar['edad'], bins=bins, labels=labels, right=False)
```

ocupado

No tiene -9 y establecemos:

- **1** si el sustentador principal de la vivienda se encuentra ocupado (recibió salario por al menos 1 hora de trabajo la semana de referencia).
- **0** en otro caso.

```
In [17]: print(datos_hogar["ocupado"].isin([-9, np.nan]).any())
```

False

```
In [18]: datos_hogar['ocupado'] = np.where(datos_hogar['ocupado'] == 1, 1, 0)
```

estudios

No tiene -9 y la categorizamos

```
In [19]: print(datos_hogar["estudios"].isin([-9, np.nan]).any())
```

False

```
In [20]: labels_estudios = {1: 'Bajo', 2: 'Bajo', 3: 'Medio', 4: 'Medio', 5: 'Medio',  
                             6: 'Alto', 7: 'Alto', 8: 'Alto'}  
orden = ["Bajo", "Medio", "Alto"]  
datos_hogar['estudios'] = pd.Categorical(datos_hogar['estudios'].map(labels_estu  
                                     ordered=True)
```

sector_publico

Tenemos -9 y NAs, por lo que primero pasamos todos a -9. Como de esos -9 incluyen a "No aplica" y datos perdidos, lo que hacemos es codificar los que no están ocupados la semana de referencia significa que no trabajan en el sector público por lo que la incluimos esas observaciones como 0 y dejamos los NAs restantes, siguiendo el siguiente criterio:

- NA si sector_publico == -9
- 1 si el sustentador principal de la vivienda trabaja en el sector_publico
- 0 en otro caso

```
In [21]: print(datos_hogar["sector_publico"].isin([-9, np.nan]).any())
```

True

```
In [22]: datos_hogar['sector_publico'] = np.where(datos_hogar['sector_publico'].isna(), -  
print(f"Datos con sector_publico == -9 antes de la codificación: {sum(datos_hoga  
datos_hogar['sector_publico'] = np.where(((datos_hogar['ocupado'] == 0)) & (dato  
datos_hogar['sector_publico'] = np.where(datos_hogar['sector_publico'] == -9, np  
print(f"Datos con sector_publico == np.nan después de la codificación: {sum(dato
```

Datos con sector_publico == -9 antes de la codificación: 3781

Datos con sector_publico == np.nan después de la codificación: 2019

ingresos_hogar

No tiene -9 y no necesita ser recodificada.

```
In [23]: print(datos_hogar["ingresos_hogar"].isin([-9, np.nan]).any())
```

False

alquiler

No tiene -9 y establecemos:

- 1 si la vivienda se encuentra en un estado de alquiler
- 0 en otro caso

```
In [24]: print(datos_hogar["alquiler"].isin([-9, np.nan]).any())
```

False

```
In [25]: datos_hogar['alquiler'] = np.where((datos_hogar['alquiler'] == 3) | (datos_hogar
```

n_habitaciones

Tiene únicamente 2 -9, por lo que eliminamos esas 2 observaciones y no hace falta recodificarla.

```
In [26]: print(datos_hogar["n_habitaciones"].isin([-9, np.nan]).any())
```

True

```
In [27]: sum(datos_hogar['n_habitaciones'] == -9)
```

Out[27]: 2

```
In [28]: datos_hogar['n_habitaciones'] = np.where(datos_hogar['n_habitaciones'] == -9, np
```

tipo_hogar

No tiene -9 y categorizamos

```
In [29]: print(datos_hogar["tipo_hogar"].isin([-9, np.nan]).any())
```

False

```
In [30]: labels_tipo_hogar = {1: 'Persona sola <65', 2: 'Persona sola ≥65', 3: 'Pareja si  
5: 'Pareja con hijos', 6: 'Pareja con hijos', 7: 'Adulto c  
datos_hogar['tipo_hogar'] = datos_hogar['tipo_hogar'].map(labels_tipo_hogar).ast
```

En gastos, tenemos un conjunto de datos con una observación por hogar y gasto específico, por lo que creamos una variable `grupo` que corresponde al grupo al que pertenece dicho gasto según

https://www.ine.es/daco/daco42/daco4213/anexo_ecoicop.pdf:

- **01** = Alimentos y bebidas no alcohólicas.
- **02** = Bebidas alcohólicas, tabaco y narcóticos.

- **03** = *Artículos de vestir y calzado.*
- **04** = *Vivienda, agua, electricidad, gas y otros combustibles.*
- **05** = *Mobiliario, equipamiento del hogar y gastos corrientes de conservación de la vivienda.*
- **06** = *Salud.*
- **07** = *Transportes.*
- **08** = *Comunicaciones.*
- **09** = *Ocio, espectáculo y cultura.*
- **10** = *Enseñanza.*
- **11** = *Hoteles, cafés y restaurantes.*
- **12** = *Otros bienes y servicios.*

Una vez clasificado cada gasto en uno de los 12 grupos principales, lo que hacemos es una transformación del conjunto de datos, agrupando cada observación por vivienda, y creando 13 variables, 12 que corresponden a la proporción de gasto de cada vivienda en cada grupo y la última que corresponde al gasto monetario en € de cada vivienda en 2023.

```
In [31]: datos_gastos["grupo"] = datos_gastos["CODIGO"].astype(str).str[:2]

group_codes = [f"{i:02d}" for i in range(1,13)]
gastos_pivot = datos_gastos.groupby(['NUMERO', 'grupo'])['GASTO'].sum().unstack(f
gastos_pivot = gastos_pivot.reindex(columns=group_codes, fill_value=0)
gasto_total = gastos_pivot.sum(axis=1)
props = gastos_pivot.div(gasto_total.replace(0, np.nan), axis=0).fillna(0)

df_gastos = props.copy()
df_gastos['gasto_total'] = gasto_total
df_gastos.insert(0, 'id', df_gastos.index)

name_map = {
    '01': 'alimentacion', '02': 'alcohol_tabaco', '03': 'ropa_calzado',
    '04': 'vivienda', '05': 'mobiliario', '06': 'salud',
    '07': 'transporte', '08': 'comunicaciones', '09': 'ocio_cultura',
    '10': 'educacion', '11': 'hoteles_restaurantes', '12': 'bienes_servicios'
}
rename_map = {code: 'prop_' + name_map[code] for code in group_codes}
df_gastos = df_gastos.rename(columns=rename_map)

df_gastos = df_gastos.reset_index(drop=True)
datos_gastos_agg = df_gastos
```

Juntamos ambas bases de datos por vivienda y nos resulta un conjunto de datos (`datos`) de 20707 hogares y 38 variables

```
In [32]: datos = datos_hogar.merge(datos_gastos_agg, on='id', how='inner')
datos.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 20707 entries, 0 to 20706
Data columns (total 30 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     20707 non-null  int64
1   factor                 20707 non-null  float64
2   c_autonoma             20707 non-null  category
3   tam_municipio          20707 non-null  category
4   tamano_ocde            20707 non-null  float64
5   n_menores              20707 non-null  int64
6   n_mayores              20707 non-null  int64
7   n_ocupados             20707 non-null  int64
8   hombre                 20707 non-null  int64
9   edad                   20707 non-null  category
10  ocupado                 20707 non-null  int64
11  estudios                20707 non-null  category
12  sector_publico          18688 non-null  float64
13  ingresos_hogar          20707 non-null  int64
14  alquiler                20707 non-null  int64
15  n_habitaciones          20705 non-null  float64
16  tipo_hogar              20707 non-null  category
17  prop_alimentacion        20707 non-null  float64
18  prop_alcohol_tabaco      20707 non-null  float64
19  prop_ropa_calzado        20707 non-null  float64
20  prop_vivienda            20707 non-null  float64
21  prop_mobiliario          20707 non-null  float64
22  prop_salud               20707 non-null  float64
23  prop_transporte          20707 non-null  float64
24  prop_comunicaciones      20707 non-null  float64
25  prop_ocio_cultura        20707 non-null  float64
26  prop_educacion           20707 non-null  float64
27  prop_hoteles_restaurantes 20707 non-null  float64
28  prop_bienes_servicios    20707 non-null  float64
29  gasto_total              20707 non-null  float64
dtypes: category(5), float64(17), int64(8)
memory usage: 4.0 MB

```

Tratamiento, depuración y construcción de la base analítica

```

In [33]: n_repetidos = (datos['id'].value_counts() > 1).sum()

print(f"Número de IDs repetidos: {n_repetidos}")

n_hogares_gasto_nulo = sum(datos['gasto_total'].isna() | (datos['gasto_total'] =
print(f"Número de hogares con gasto total nulo o faltante: {n_hogares_gasto_nulo}

```

Número de IDs repetidos: 0
 Número de hogares con gasto total nulo o faltante: 0

```

In [34]: print("Número de valores nulos en 'n_habitaciones':", round(sum(datos['n_habitac
print("Proporción de valores nulos en 'n_habitaciones':", round(sum(datos['n_hab
datos = datos[~datos['n_habitaciones'].isna()])
print("Número de valores nulos en 'n_habitaciones' después de eliminarlos:", rou
print("Proporción de valores nulos en 'n_habitaciones' después de eliminarlos:",

```

Número de valores nulos en 'n_habitaciones': 2
Proporción de valores nulos en 'n_habitaciones': 0.0001
Número de valores nulos en 'n_habitaciones' después de eliminarlos: 0
Proporción de valores nulos en 'n_habitaciones' después de eliminarlos: 0.0

```
In [35]: print("Número de valores nulos en 'sector_publico':", round(sum(datos['sector_publico'].isnull()), 2))  
print("Proporción de valores nulos en 'sector_publico':", round(sum(datos['sector_publico'].isnull()) / len(datos['sector_publico']), 2))
```

Número de valores nulos en 'sector_publico': 2019
Proporción de valores nulos en 'sector_publico': 0.09751

Análisis descriptivo y exploratorio

Contexto económico del hogar

- **factor** (factor de elevación): peso muestral asignado por el INE a cada hogar encuestado. Indica cuántos hogares de la población representa. Se emplea para obtener estimaciones poblacionales; **en este EDA no se ponderan los estadísticos**, por lo que todos los resultados son descriptivos muestrales.
- **gasto_total** = gasto monetario anual (€) del hogar, **ya multiplicado** por el factor de elevación en la fuente original.
- **gasto_total_crudo** = $\frac{\text{gasto_total}}{\text{factor}}$, es decir, el gasto anual *muestral* (sin ponderar) del hogar en euros corrientes de 2023.
- **ingresos_hogar** = ingresos netos mensuales del hogar declarados en la encuesta.

```
In [36]: datos['gasto_total_crudo'] = datos['gasto_total'] / datos['factor']  
datos['log_gasto_total_crudo'] = np.log1p(datos['gasto_total_crudo'])  
datos['log_ingresos_hogar'] = np.log1p(datos['ingresos_hogar'])  
  
variables_mon = ['gasto_total_crudo', 'ingresos_hogar']  
  
summary_rows = []  
for v in variables_mon:  
    s = datos[v]  
    row = {  
        'Variable': v,  
        'Media': round(s.mean(), 2),  
        'DE': round(s.std(), 2),  
        'Min': round(s.min(), 2),  
        'P25': round(s.quantile(0.25), 2),  
        'Mediana': round(s.median(), 2),  
        'P75': round(s.quantile(0.75), 2),  
        'Max': round(s.max(), 2)  
    }  
    summary_rows.append(row)  
  
tabla_monetaria = pd.DataFrame(summary_rows)  
  
display(tabla_monetaria)
```


	Variable	Media	DE	Min	P25	Mediana	P75	Max
0	gasto_total_crudo	33329.55	19458.52	1106.78	19484.61	28964.36	42196.47	272006.1
1	ingresos_hogar	2514.83	1534.22	0.00	1390.00	2244.00	3218.00	36400.0

```
In [37]: variables = ["ingresos_hogar", "gasto_total_crudo"]
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

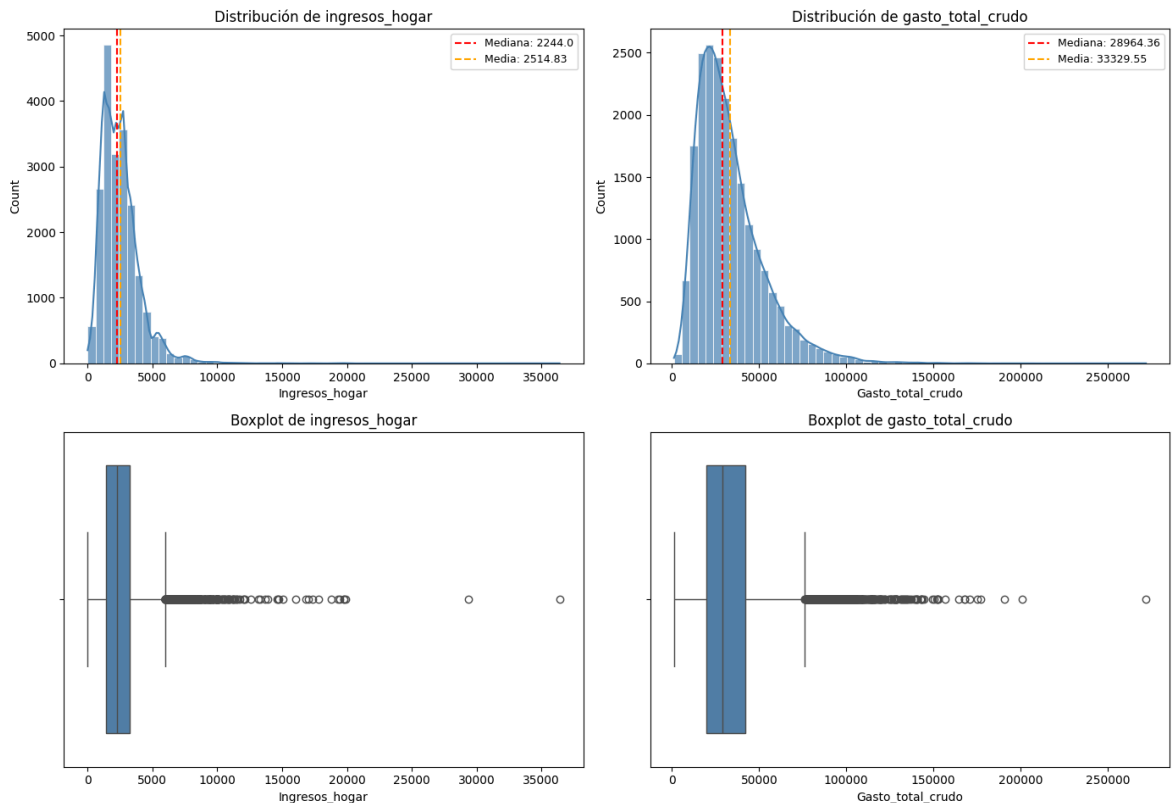
for i, var in enumerate(variables):
    s = datos[var]
    ax_hist = axes[0, i]
    sns.histplot(s, bins=60, kde=True, color='steelblue', edgecolor='white', alp
    ax_hist.axvline(s.median(), color='red', ls='--', lw=1.5, label=f'Mediana: {
    ax_hist.axvline(s.mean(), color='orange', ls='--', lw=1.5, label=f'Media: {r

    ax_hist.set_title(f'Distribución de {var}')
    ax_hist.set_xlabel(var.capitalize())
    ax_hist.legend(fontsize=9)

    ax_box = axes[1, i]
    sns.boxplot(x=s, color='steelblue', ax=ax_box)
    ax_box.set_xlabel(var.capitalize())
    ax_box.set_title(f'Boxplot de {var}')

plt.suptitle('Distribuciones de variables monetarias (escala original)', fontsiz
plt.tight_layout()
plt.show()
```

Distribuciones de variables monetarias (escala original)



```
In [38]: variables = ["log_ingresos_hogar", "log_gasto_total_crudo"]
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

for i, var in enumerate(variables):
```

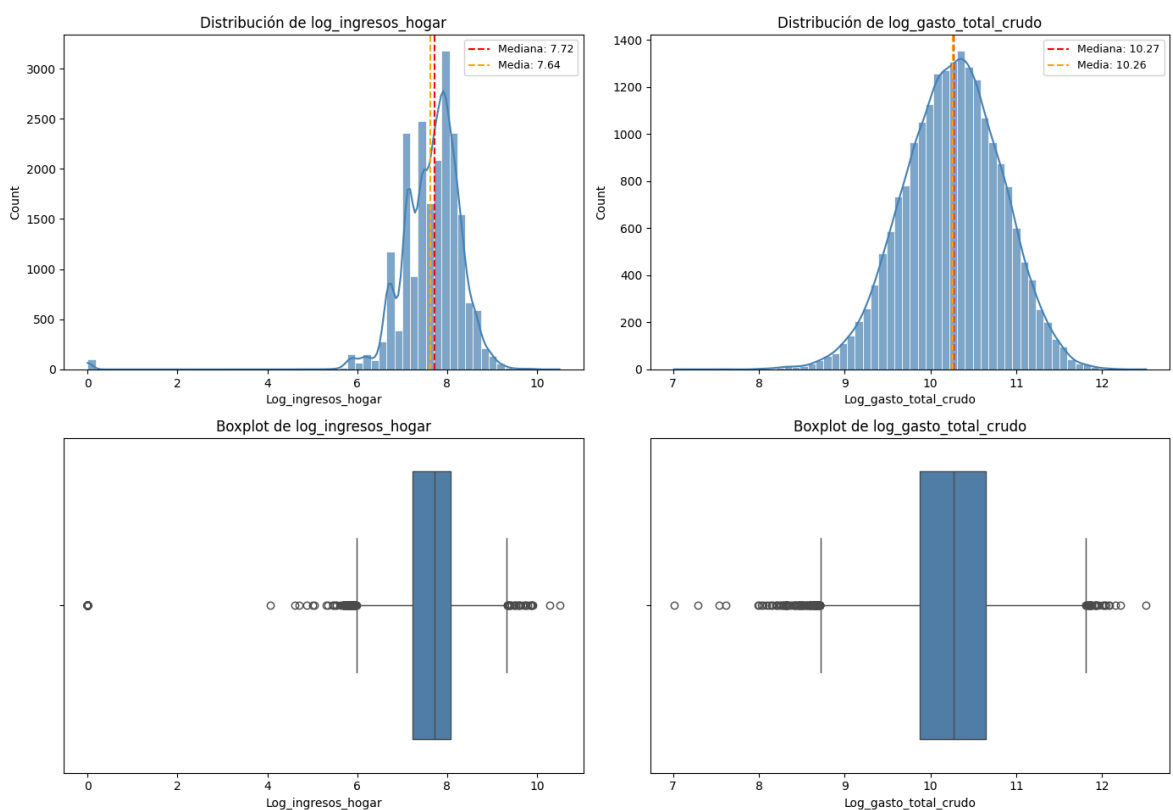
```

s = datos[var]
ax_hist = axes[0, i]
sns.histplot(s, bins=60, kde=True, color='steelblue', edgecolor='white', alp
ax_hist.axvline(s.median(), color='red', ls='--', lw=1.5, label=f'Mediana: {r
ax_hist.axvline(s.mean(), color='orange', ls='--', lw=1.5, label=f'Media: {r
ax_hist.set_title(f'Distribución de {var}')
ax_hist.set_xlabel(var.capitalize())
ax_hist.legend(fontsize=9)
ax_box = axes[1, i]
sns.boxplot(x=s, color='steelblue', ax=ax_box)
ax_box.set_xlabel(var.capitalize())
ax_box.set_title(f'Boxplot de {var}')

plt.suptitle('Distribuciones de variables monetarias (escala original)', fontsiz
plt.tight_layout()
plt.show()

```

Distribuciones de variables monetarias (escala original)



Distribución univariante de las proporciones de gasto

```

In [39]: proporciones = [col for col in datos.columns if col.startswith('prop_')]
rows_p = []
for p in proporciones:
    s = datos[p]
    rows_p.append({
        'Variable': p.replace('prop_', ''),
        '% hogares con gasto 0': round((s == 0).mean() * 100, 1),
        'Media': round(s.mean(), 4),
        'Mediana': round(s.median(), 4),
        'IQR': round(s.quantile(0.75) - s.quantile(0.25), 4)
    })

tabla_props = pd.DataFrame(rows_p)

```

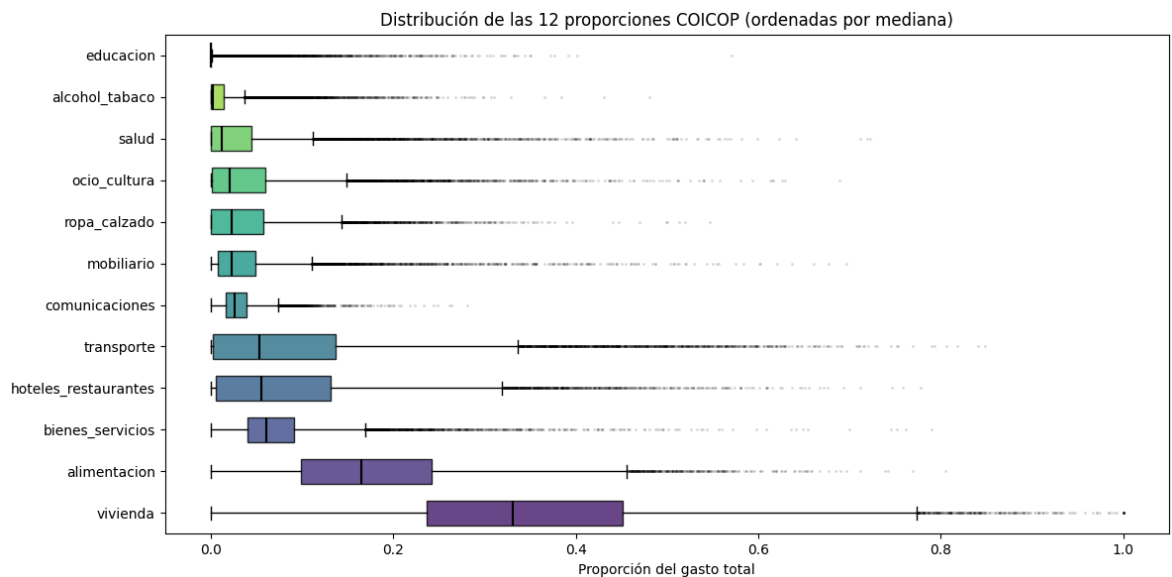
```
display(tabla_props)
```

	Variable	% hogares con gasto 0	Media	Mediana	IQR
0	alimentacion	2.7	0.1784	0.1653	0.1430
1	alcohol_tabaco	43.4	0.0148	0.0023	0.0147
2	ropa_calzado	26.7	0.0384	0.0231	0.0572
3	vivienda	0.0	0.3563	0.3310	0.2148
4	mobiliario	8.6	0.0381	0.0231	0.0410
5	salud	29.6	0.0360	0.0125	0.0448
6	transporte	15.4	0.0930	0.0532	0.1336
7	comunicaciones	3.0	0.0309	0.0261	0.0231
8	ocio_cultura	24.0	0.0438	0.0211	0.0590
9	educacion	74.2	0.0101	0.0000	0.0003
10	hoteles_restaurantes	23.4	0.0863	0.0555	0.1256
11	bienes_servicios	1.1	0.0738	0.0612	0.0517

```
In [40]: medians_order = datos[proporciones].median().sort_values(ascending=False)
ordered_props = medians_order.index.tolist()
labels_short = [p.replace('prop_', '') for p in ordered_props]

fig, ax = plt.subplots(figsize=(12, 6))
bp = ax.boxplot(
    [datos[p].dropna() for p in ordered_props],
    vert=False, patch_artist=True, widths=0.6,
    flierprops=dict(marker='.', markersize=2, alpha=0.15, color='gray'),
    medianprops=dict(color='black', linewidth=1.5)
)
colors = sns.color_palette("viridis", len(ordered_props))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.8)

ax.set_yticks(range(1, len(labels_short)+1))
ax.set_yticklabels(labels_short)
ax.set_xlabel('Proporción del gasto total')
ax.set_title(f'Distribución de las 12 proporciones COICOP (ordenadas por mediana)')
plt.tight_layout()
plt.show()
```

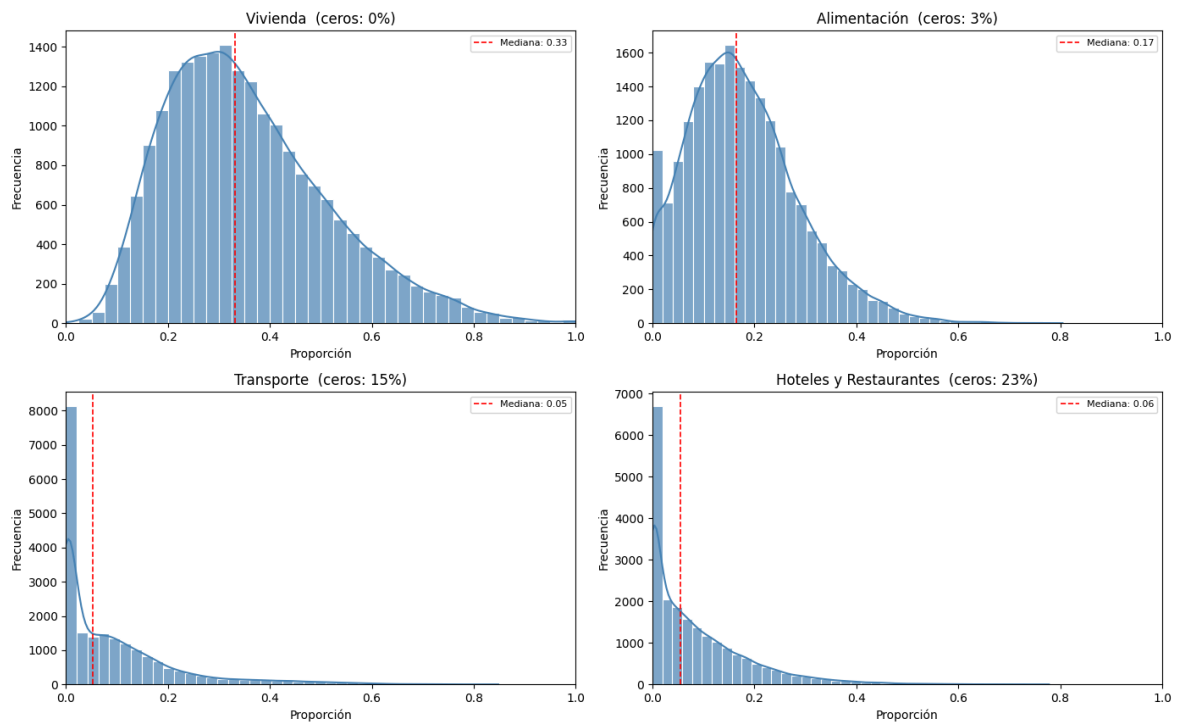


```
In [41]: cols_hist = ['prop_vivienda', 'prop_alimentacion', 'prop_transporte', 'prop_hoteles_restaurantes']
titles_hist = ['Vivienda', 'Alimentación', 'Transporte', 'Hoteles y Restaurantes']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
for ax, col, title in zip(axes.ravel(), cols_hist, titles_hist):
    s = datos[col]
    sns.histplot(s, bins=40, kde=True, ax=ax, color='steelblue', edgecolor='white')
    ax.set_xlim(0, 1)
    ax.set_xlabel('Proporción')
    ax.set_ylabel('Frecuencia')
    pct_zero = (s == 0).mean() * 100
    skew = s.skew()
    ax.set_title(f'{title} (ceros: {pct_zero:.0f}%)')
    ax.axvline(s.median(), color='red', ls='--', lw=1.2, label=f'Mediana: {round(s.median(), 2)}')
    ax.legend(fontsize=8)

plt.suptitle(f'Histogramas de proporciones COICOP representativas', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

Histogramas de proporciones COICOP representativas

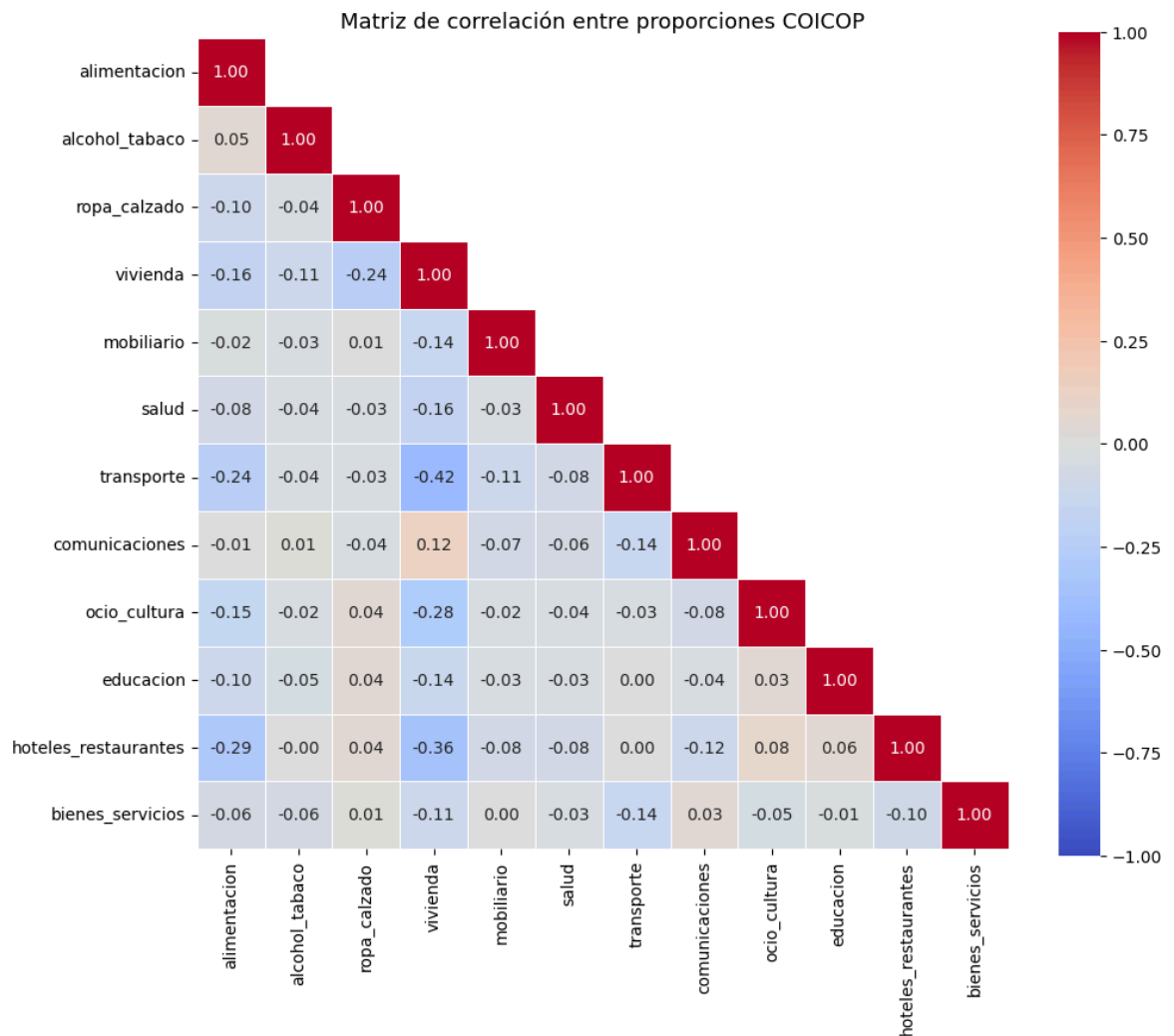


Naturaleza composicional del presupuesto

```
In [42]: corr_matrix = datos[proporciones].corr()
labels_c = [p.replace('prop_', '') for p in proporciones]

fig, ax = plt.subplots(figsize=(11, 9))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1,
            xticklabels=labels_c, yticklabels=labels_c, mask=mask, ax=ax,
            linewidths=0.5, square=True)
ax.set_title(f'Matriz de correlación entre proporciones COICOP', fontsize=13)
```

```
Out[42]: Text(0.5, 1.0, 'Matriz de correlación entre proporciones COICOP')
```

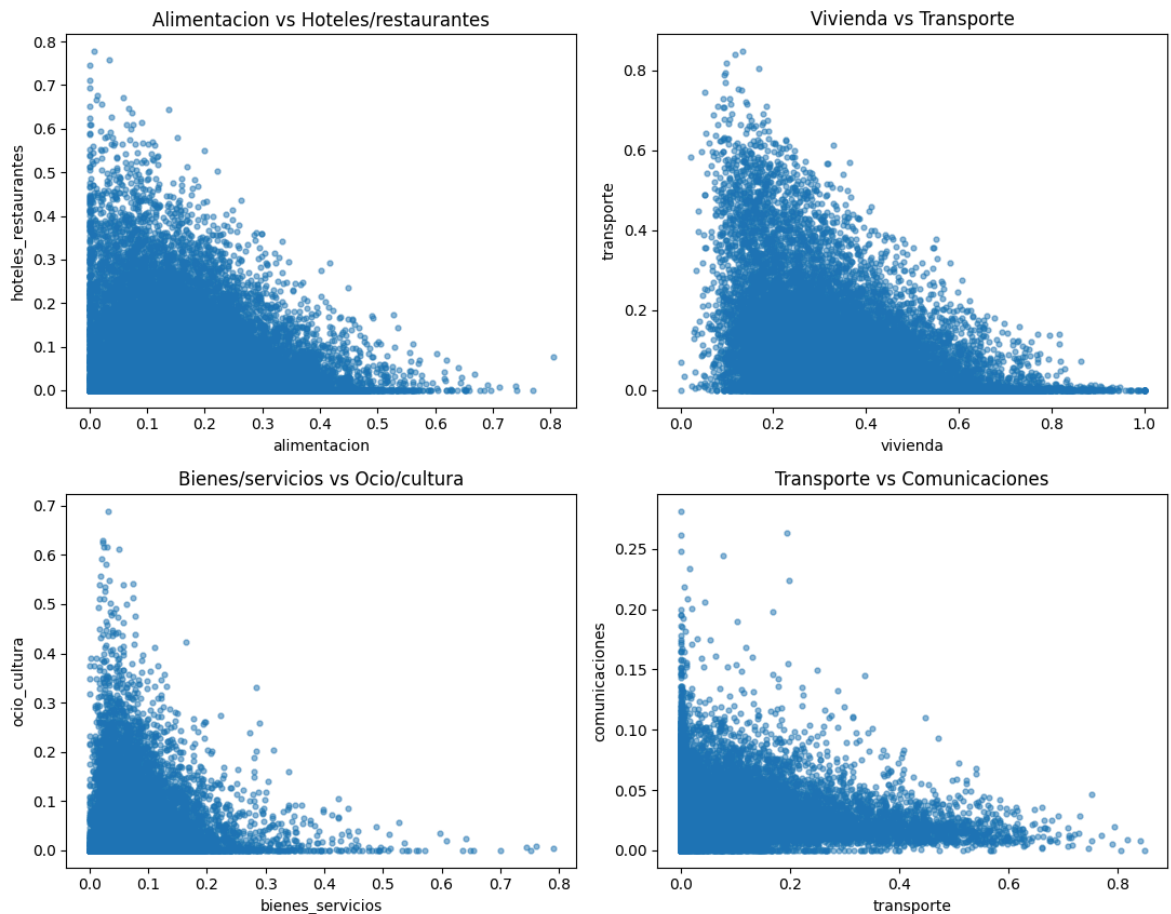


```
In [43]: pairs = [
    ('prop_alimentacion', 'prop_hoteles_restaurantes', 'Alimentacion vs Hoteles/'),
    ('prop_vivienda', 'prop_transporte', 'Vivienda vs Transporte'),
    ('prop_bienes_servicios', 'prop_ocio_cultura', 'Bienes/servicios vs Ocio/cul'),
    ('prop_transporte', 'prop_comunicaciones', 'Transporte vs Comunicaciones'),
]

fig, axes = plt.subplots(2, 2, figsize=(11, 9))
axes = axes.ravel()
for i, (x, y, title) in enumerate(pairs):
    ax = axes[i]
    ax.scatter(datos[x], datos[y], s=12, alpha=0.5)
    ax.set_xlabel(x.replace('prop_', ''))
    ax.set_ylabel(y.replace('prop_', ''))
    ax.set_title(title)

plt.suptitle('Diagramas de dispersion de parejas seleccionadas', fontsize=13, y=
plt.tight_layout()
plt.show()
```

Diagramas de dispersion de parejas seleccionadas



Descripción de las variables sociodemográficas

```
In [44]: cont_vars = ['tamano_ocde', 'n_menores', 'n_mayores', 'n_ocupados', 'n_habitacio
rows_c = []
for v in cont_vars:
    s = datos[v].dropna()
    rows_c.append({
        'Variable': v,
        'Media': round(s.mean(), 2),
        'Desviacion': round(s.std(), 2),
        'Min': round(s.min(), 2),
        'P25': round(s.quantile(0.25), 2),
        'Mediana': round(s.median(), 2),
        'p75': round(s.quantile(0.75), 2),
        'Max': round(s.max(), 2),
    })

tabla_numerica = pd.DataFrame(rows_c)
display(tabla_numerica)
```

	Variable	Media	Desviacion	Min	P25	Mediana	p75	Max
0	tamano_ocde	1.99	0.78	1.0	1.7	1.7	2.4	7.6
1	n_menores	0.44	0.79	0.0	0.0	0.0	1.0	8.0
2	n_mayores	2.06	0.89	1.0	1.0	2.0	2.0	9.0
3	n_ocupados	1.08	0.91	0.0	0.0	1.0	2.0	6.0
4	n_habitaciones	5.07	1.18	1.0	4.0	5.0	6.0	8.0

```
In [45]: fig, ax = plt.subplots(figsize=(10, 6))

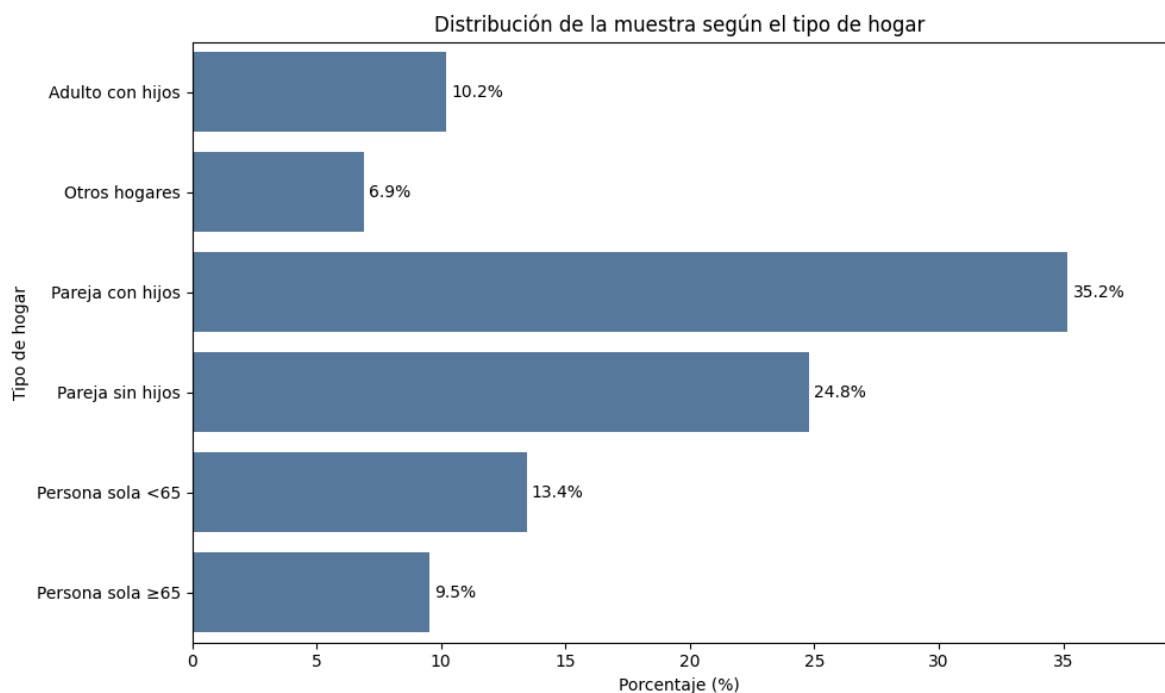
color_hogar = "#4C78A8"

p1 = datos["tipo_hogar"].dropna().value_counts(normalize=True).mul(100)

sns.barplot(x=p1.values, y=p1.index, ax=ax, color=color_hogar)
ax.set_title("Distribución de la muestra según el tipo de hogar")
ax.set_xlabel("Porcentaje (%)")
ax.set_ylabel("Tipo de hogar")
ax.set_xlim(0, p1.max() + 4)

for barra in ax.patches:
    ancho = barra.get_width()
    y = barra.get_y() + barra.get_height() / 2
    ax.text(ancho + 0.2, y, f"{ancho:.1f}%", va="center", ha="left")

plt.tight_layout()
plt.show()
```



```
In [46]: fig, ax = plt.subplots(2, 2, figsize=(14, 10), gridspec_kw={"height_ratios": [2,
```



```

orden_edad = (
    list(datos["edad"].cat.categories)
    if pd.api.types.is_categorical_dtype(datos["edad"])
    else list(datos["edad"].dropna().unique())
)
p_edad = datos["edad"].dropna().value_counts(normalize=True).mul(100).reindex(orden_edad)

sns.barplot(x=p_edad.index, y=p_edad.values, ax=ax[0, 0], color=color_edad)
ax[0, 0].set_title("Edad por tramos")
ax[0, 0].set_xlabel("")
ax[0, 0].set_ylabel("Porcentaje (%)")
ax[0, 0].tick_params(axis="x", rotation=35)
for barra in ax[0, 0].patches:
    alto = barra.get_height()
    x = barra.get_x() + barra.get_width() / 2
    ax[0, 0].text(x, alto + 0.3, f"{alto:.1f}%", ha="center", va="bottom")

orden_estudios = (
    list(datos["estudios"].cat.categories)
    if pd.api.types.is_categorical_dtype(datos["estudios"])
    else list(datos["estudios"].dropna().unique())
)
p_estudios = datos["estudios"].dropna().value_counts(normalize=True).mul(100).reindex(orden_estudios)

sns.barplot(x=p_estudios.index, y=p_estudios.values, ax=ax[0, 1], color=color_estudios)
ax[0, 1].set_title("Nivel de estudios")
ax[0, 1].set_xlabel("")
ax[0, 1].set_ylabel("Porcentaje (%)")
ax[0, 1].tick_params(axis="x", rotation=35)
for barra in ax[0, 1].patches:
    alto = barra.get_height()
    x = barra.get_x() + barra.get_width() / 2
    ax[0, 1].text(x, alto + 0.3, f"{alto:.1f}%", ha="center", va="bottom")

p_ocupado = datos["ocupado"].dropna().value_counts(normalize=True).mul(100).sort_index()
p_ocupado.index = p_ocupado.index.map({0: "No", 1: "Si"})

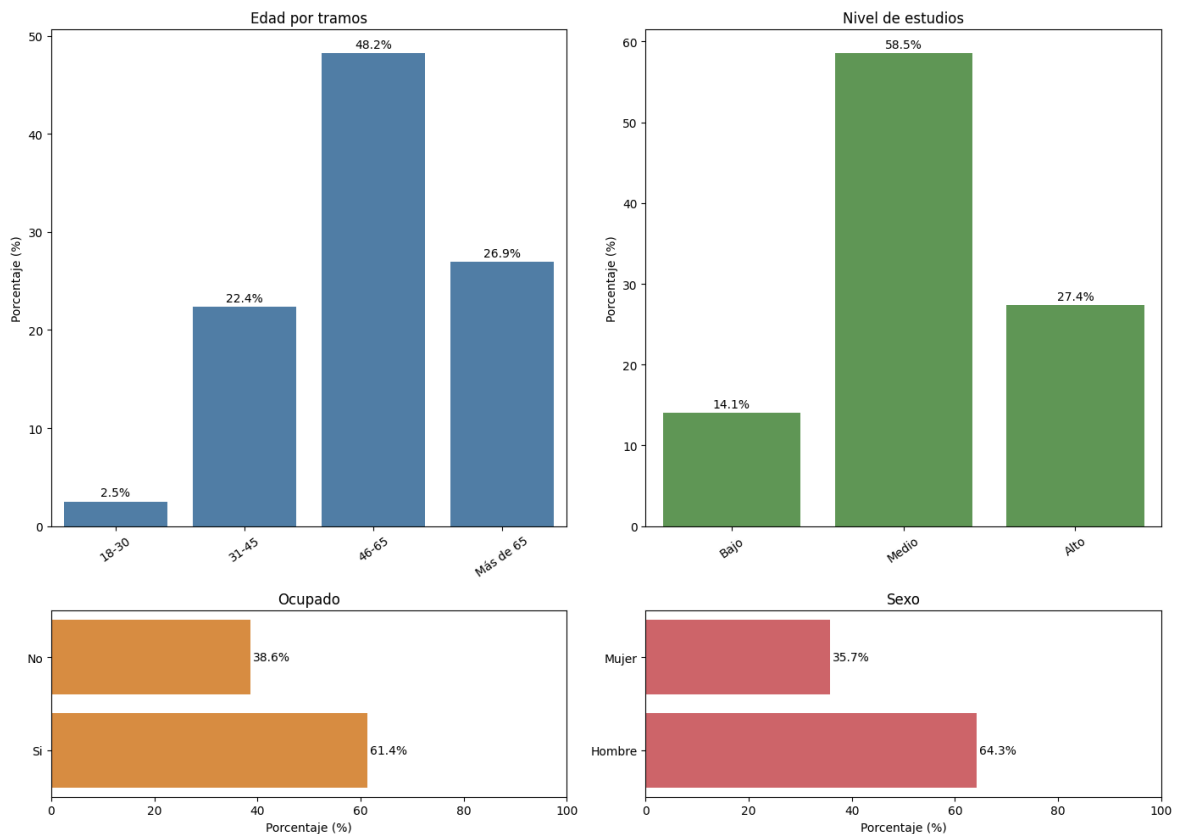
sns.barplot(x=p_ocupado.index, y=p_ocupado.values, ax=ax[1, 0], color=color_ocupado)
ax[1, 0].set_title("Ocupado")
ax[1, 0].set_xlabel("Porcentaje (%)")
ax[1, 0].set_ylabel("")
ax[1, 0].set_xlim(0, 100)
for barra in ax[1, 0].patches:
    ancho = barra.get_width()
    y = barra.get_y() + barra.get_height() / 2
    ax[1, 0].text(ancho + 0.5, y, f"{ancho:.1f}%", va="center", ha="left")

p_hombre = datos["hombre"].dropna().value_counts(normalize=True).mul(100).sort_index()
p_hombre.index = p_hombre.index.map({0: "Mujer", 1: "Hombre"})

sns.barplot(x=p_hombre.index, y=p_hombre.values, ax=ax[1, 1], color=color_hombre)
ax[1, 1].set_title("Sexo")
ax[1, 1].set_xlabel("Porcentaje (%)")
ax[1, 1].set_ylabel("")
ax[1, 1].set_xlim(0, 100)
for barra in ax[1, 1].patches:
    ancho = barra.get_width()
    y = barra.get_y() + barra.get_height() / 2
    ax[1, 1].text(ancho + 0.5, y, f"{ancho:.1f}%", va="center", ha="left")

```

```
plt.tight_layout()
plt.show()
```



Cruces exploratorios

```
In [47]: data = datos.copy()

data["alquiler_label"] = data["alquiler"].map({0: "No", 1: "Sí"})
data["ocupado_label"] = data["ocupado"].map({0: "No", 1: "Sí"})

def plot_mediana_iqr(ax, y, grupo, titulo, xlabel=""):
    resumen = (
        data[[y, grupo]]
        .dropna()
        .groupby(grupo)[y]
        .agg(
            q1=lambda x: x.quantile(0.25),
            mediana="median",
            q3=lambda x: x.quantile(0.75)
        )
        .reset_index()
    )

    x = range(len(resumen))

    ax.errorbar(
        x,
        resumen["mediana"],
        yerr=[
            resumen["mediana"] - resumen["q1"],
            resumen["q3"] - resumen["mediana"]
        ],
        fmt="o",
```

```

        capsize=5
    )

    ax.set_xticks(x)
    ax.set_xticklabels(resumen[grupo], rotation=35, ha="right")
    ax.set_title(titulo)
    ax.set_ylabel("Proporción del gasto total")
    ax.set_xlabel(xlabel)
    ax.set_ylim(bottom=0)
    ax.grid(axis="y", alpha=0.3)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

plot_mediana_iqr(
    axes[0, 0],
    y="prop_vivienda",
    grupo="alquiler_label",
    titulo="Vivienda según régimen de alquiler"
)

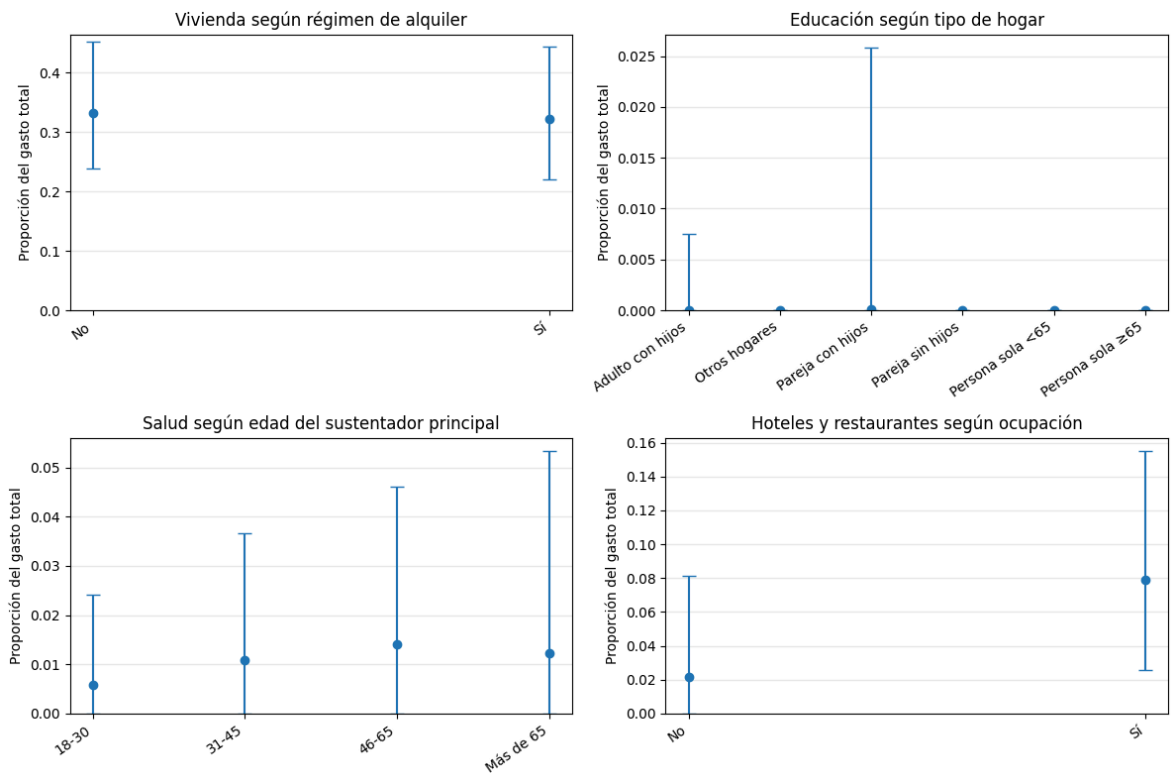
plot_mediana_iqr(
    axes[0, 1],
    y="prop_educacion",
    grupo="tipo_hogar",
    titulo="Educación según tipo de hogar"
)

plot_mediana_iqr(
    axes[1, 0],
    y="prop_salud",
    grupo="edad",
    titulo="Salud según edad del sustentador principal"
)

plot_mediana_iqr(
    axes[1, 1],
    y="prop_hoteles_restaurantes",
    grupo="ocupado_label",
    titulo="Hoteles y restaurantes según ocupación"
)

plt.tight_layout()
plt.show()

```



```
In [48]: datos['quintil_renta'] = pd.qcut(datos['ingresos_hogar'], q=5,
                                           labels=['Q1 (más baja)', 'Q2', 'Q3', 'Q4', 'Q5'])

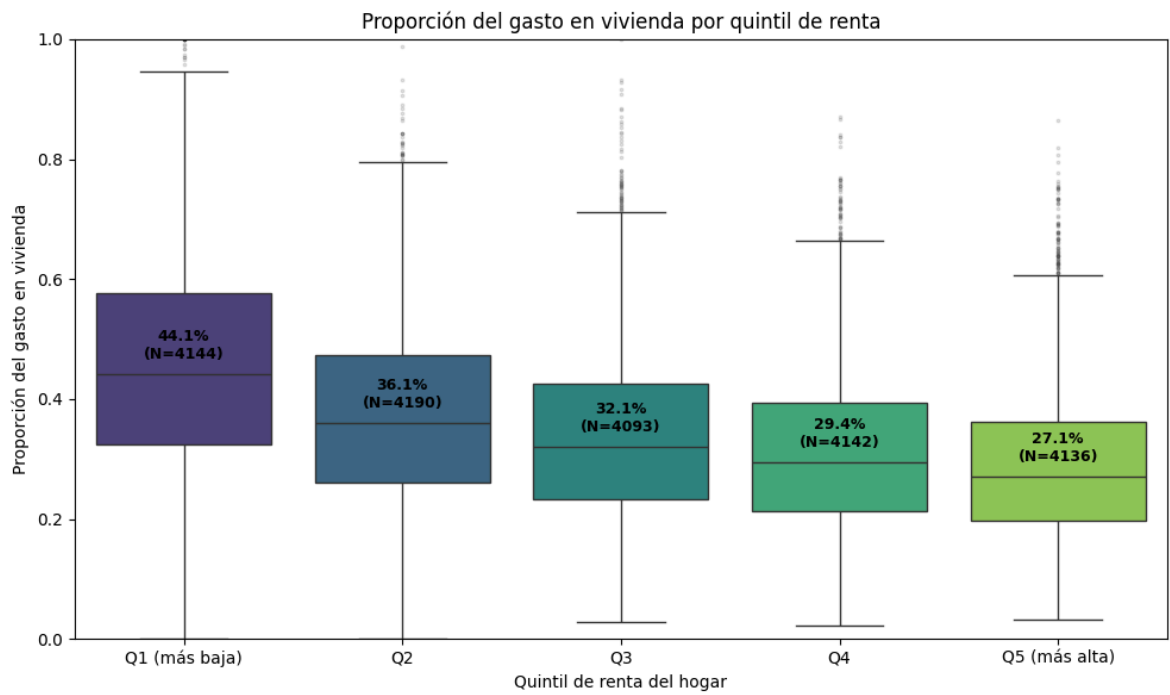
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(x='quintil_renta', y='prop_vivienda', data=datos, palette='viridis',
            flierprops=dict(marker='.', markersize=3, alpha=0.2))

medians = datos.groupby('quintil_renta', observed=True)['prop_vivienda'].median()
ns = datos.groupby('quintil_renta', observed=True).size()
for i, (med, n) in enumerate(zip(medians, ns)):
    ax.text(i, med + 0.02, f'{med:.1%}\n(N={n})', ha='center', va='bottom', font

ax.set_xlabel('Quintil de renta del hogar')
ax.set_ylabel('Proporción del gasto en vivienda')
ax.set_title(f'Proporción del gasto en vivienda por quintil de renta')
ax.set_ylim(0, 1)
plt.tight_layout()
plt.show()

tabla_q = datos.groupby('quintil_renta', observed=True)['prop_vivienda'].agg(
    N='count', Media=lambda x: round(x.mean(), 4), Mediana='median',
    IQR=lambda x: round(x.quantile(0.75) - x.quantile(0.25), 4)
).round(4)

del(datos['quintil_renta'])
print("Proporción de gasto en vivienda por quintil de renta")
display(tabla_q)
```



Proporción de gasto en vivienda por quintil de renta

	N	Media	Mediana	IQR
quintil_renta				
Q1 (más baja)	4144	0.4571	0.4409	0.2523
Q2	4190	0.3769	0.3607	0.2139
Q3	4093	0.3412	0.3207	0.1918
Q4	4142	0.3141	0.2939	0.1807
Q5 (más alta)	4136	0.2918	0.2711	0.1645

```
In [49]: datos.to_csv('../datos_procesados/datos_iniciales.csv', index=False)
```